Assignment 3

Name : Om A. Akiwate
PRN : 255209003
BATCH : BT4

Dataset — Generate It Yourself

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(seed=42)   # fixed — do not change
n   = 200

# Normal distribution: systolic blood pressure (mm Hg)
bp   = rng.normal(loc=120, scale=15, size=n).round(1)

# Gamma distribution: patient waiting time (minutes)
wait = rng.gamma(shape=2, scale=30, size=n).round(1)

# Inject 5 high outliers into each column
bp[rng.choice(n, 5, replace=False)]   = rng.uniform(190, 230, 5).round(1)
wait[rng.choice(n, 5, replace=False)] = rng.uniform(350, 500, 5).round(1)

df = pd.DataFrame({"bp": bp, "wait": wait})
df.to_csv("normalisation_data.csv", index=False)
print(df.describe().round(2))
df.head()
```

```
...              bp    wait
count  200.00  200.00
mean   121.72   72.46
std     18.72   74.97
min     88.00    2.80
25%    110.40   32.38
50%    119.80   54.75
75%    129.43   87.48
max    221.10  488.70
```

...

	bp	wait
0	124.6	64.2
1	104.4	53.6
2	131.3	5.2
3	134.1	23.9
4	90.7	23.1

A.2 Python One-Liners for Each Technique Let s = df["bp"] (a pandas Series). Each line produces a new Series — assign it to a variable.

```
s = df["bp"]                                    # or df["wait"]

bp_min  = s - s.min()                           # min scaling
bp_max  = s / s.max()                           # max scaling
bp_mm   = (s - s.min()) / (s.max() - s.min())   # min-max
bp_z    = (s - s.mean()) / s.std()              # z-score

Q1, Q3  = s.quantile(0.25), s.quantile(0.75)
bp_iqr  = (s - s.median()) / (Q3 - Q1)          # IQR / robust
```

✓ 0.0s

A.3 Plotting: Histogram + Box Plot Side by Side After Each Normalisation Use this template after every normalisation. Replace bp_mm and the title string each time.

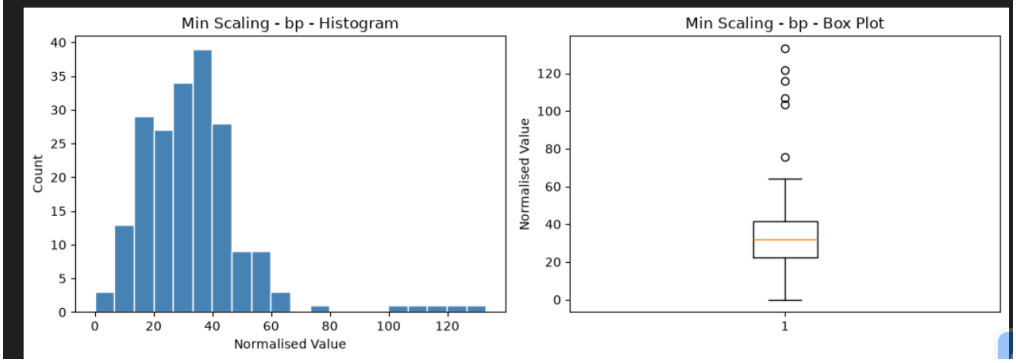
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_min, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Min Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_min)
ax2.set_title("Min Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.2s



2.Max Scaling

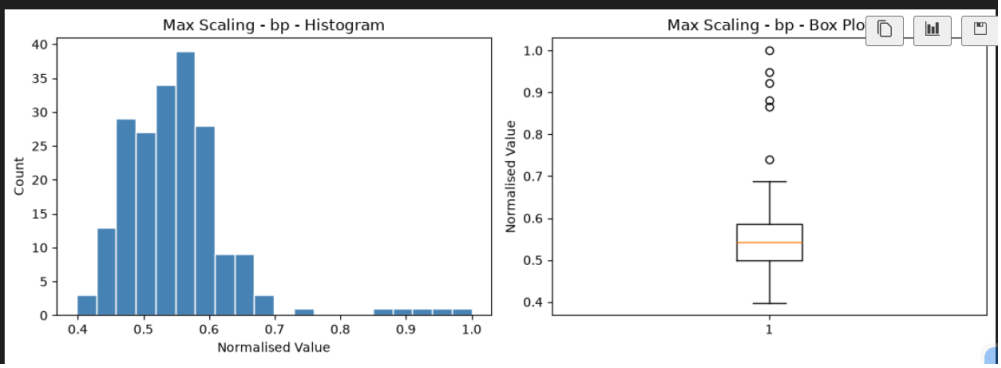
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_max, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Max Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_max)
ax2.set_title("Max Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



3.Min-Max Scaling

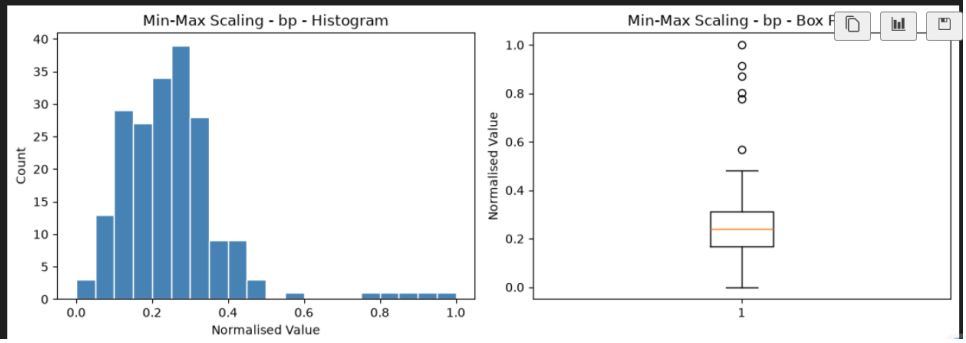
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_mm, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Min-Max Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_mm)
ax2.set_title("Min-Max Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



4.Z-Score Scaling

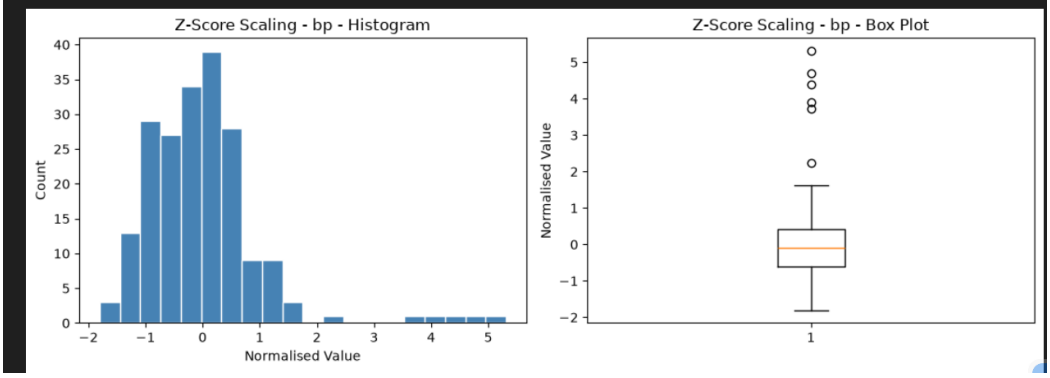
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_z, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Z-Score Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_z)
ax2.set_title("Z-Score Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



5.Robust(IQR) Scaling

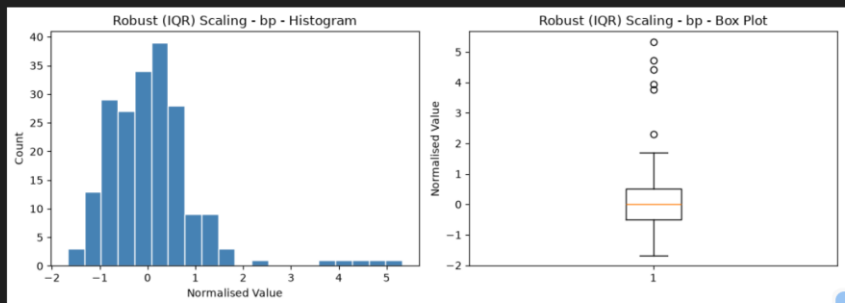
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_iqr, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Robust (IQR) Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_iqr)
ax2.set_title("Robust (IQR) Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.2s



A4. Quick Statistics After Normalisation

The summary statistics of the normalised data are calculated to compare how each normalisation technique changes the scale and distribution of the original data.

B.1 Generate and Explore the Raw Data

Run the dataset cell from Section 0, then work only with normalisation_data.csv from here on.

B1. Load normalisation_data.csv. Print shape, head, and describe(). In a markdown cell, record for each column: the mean, median, min, max, and whether mean is above or below median. Explain what a large gap between mean and median tells you about shape.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("normalisation_data.csv")

print("Shape:", df.shape)

print("\nFirst 5 rows:")
print(df.head())

print("\nSummary statistics:")
print(df.describe().round(2))
```

✓ 0.0s

```
Shape: (200, 2)

First 5 rows:
      bp  wait
0  124.6  64.2
1  104.4  53.6
2  131.3   5.2
3  134.1  23.9
4   90.7  23.1

Summary statistics:
             bp    wait
count  200.00  200.00
mean   121.72   72.46
std     18.72   74.97
min     88.00    2.80
25%    110.40   32.38
50%    119.80   54.75
75%    129.43   87.48
max    221.10  488.70
```

B2. Plot a histogram and box plot of the raw bp column side by side. Repeat for the raw wait column. In a markdown cell, compare the two pairs of plots — which column looks more symmetric, and which shows a longer upper tail? Are outlier dots visible in the box plots?

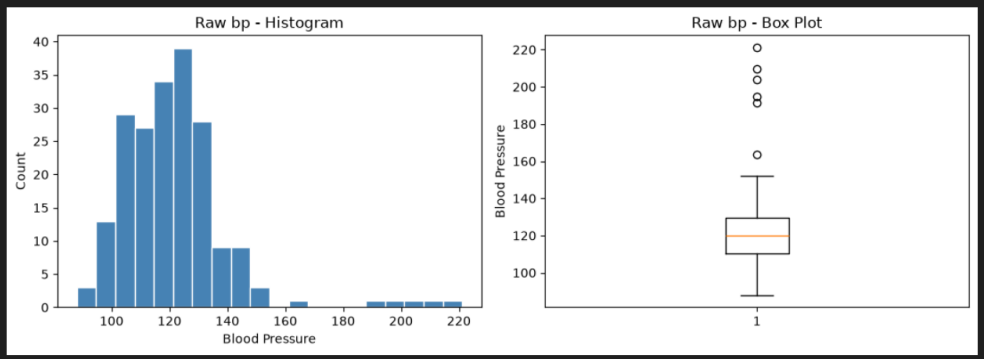
```
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(df["bp"], bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Raw bp - Histogram")
ax1.set_xlabel("Blood Pressure")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(df["bp"])
ax2.set_title("Raw bp - Box Plot")
ax2.set_ylabel("Blood Pressure")

plt.tight_layout()
plt.show()
```

✓ 0.1s



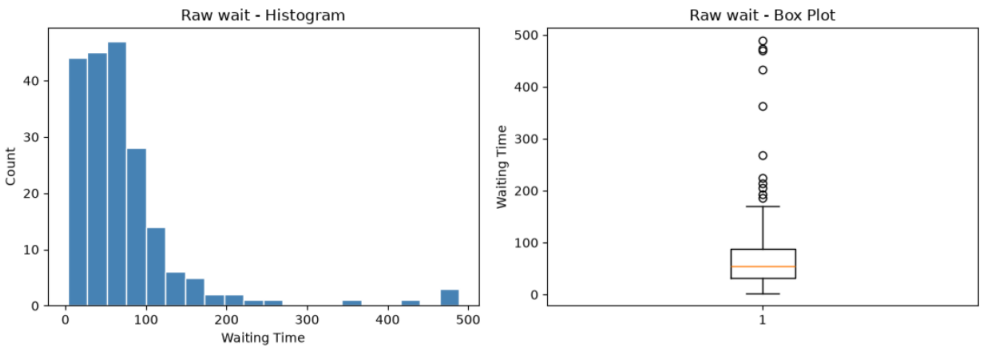

```
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(df["wait"], bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Raw wait - Histogram")
ax1.set_xlabel("Waiting Time")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(df["wait"])
ax2.set_title("Raw wait - Box Plot")
ax2.set_ylabel("Waiting Time")

plt.tight_layout()
plt.show()
```

✓ 0.1s



B3. Compute the skewness of both raw columns using pd.Series.skew(). Record the values in a markdown cell and explain what positive skewness means visually. Which column is more skewed, and does that match what you see in the plots?

```
bp_skew = df["bp"].skew()
wait_skew = df["wait"].skew()

print("Skewness of bp   :", round(bp_skew, 3))
print("Skewness of wait :", round(wait_skew, 3))
```

✓ 0.0s

```
Skewness of bp   : 2.203
Skewness of wait : 3.445
```

B.2 Min Scaling (x − min)

B4. Apply min scaling to both columns. Print the min and max of each result and confirm that every minimum is now exactly 0.

```
bp_min = df["bp"] - df["bp"].min()
wait_min = df["wait"] - df["wait"].min()

print("BP Min Scaling")
print("Min:", bp_min.min())
print("Max:", bp_min.max())

print("\nWait Min Scaling")
print("Min:", wait_min.min())
print("Max:", wait_min.max())
```

✓ 0.0s

```
BP Min Scaling
Min: 0.0
Max: 133.1

Wait Min Scaling
Min: 0.0
Max: 485.9
```

B5. For each column, plot a histogram and box plot side by side (label the technique and column in the title). Write one sentence per column: what changed on the x-axis compared to the raw plots, and what stayed the same

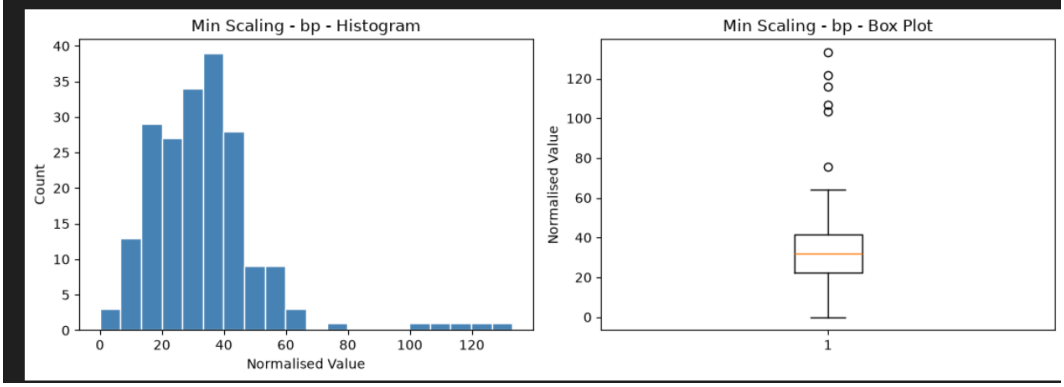
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.hist(bp_min, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Min Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

ax2.boxplot(bp_min)
ax2.set_title("Min Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.3s



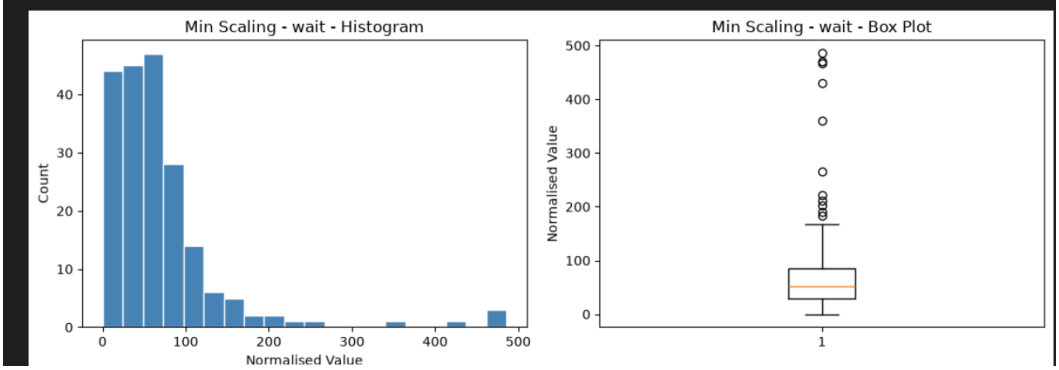

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.hist(wait_min, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Min Scaling - wait - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

ax2.boxplot(wait_min)
ax2.set_title("Min Scaling - wait - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.3s



B6. Observation

Min Scaling is useful when we want all values to start from 0 while keeping the original distribution shape. One limitation is that it does not set the maximum to a fixed value and does not reduce the effect of outliers.

B.3 Max Scaling (x / max)

B7. Apply max scaling to both columns. Print the min and max of each result. Is the minimum of bp_max the same as the minimum of bp_mm? Why or why not?

```
bp_max = df["bp"] / df["bp"].max()
wait_max = df["wait"] / df["wait"].max()

print("BP Max Scaling")
print("Min:", round(bp_max.min(), 4))
print("Max:", round(bp_max.max(), 4))

print("\nWait Max Scaling")
print("Min:", round(wait_max.min(), 4))
print("Max:", round(wait_max.max(), 4))
```

✓ 0.0s

```
BP Max Scaling
Min: 0.398
Max: 1.0

Wait Max Scaling
Min: 0.0057
Max: 1.0
```

B8. Plot histogram + box plot for each column after max scaling. In a markdown cell, compare these plots with the min scaling plots from B.2: what is different, and why does the minimum of the max-scaled column not equal 0?

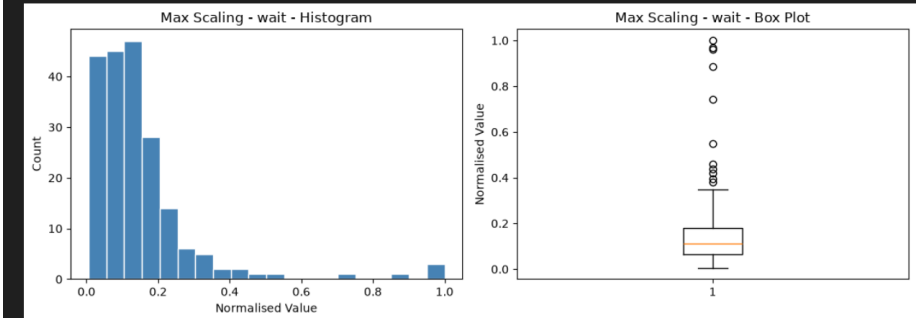
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(wait_max, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Max Scaling - wait - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(wait_max)
ax2.set_title("Max Scaling - wait - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



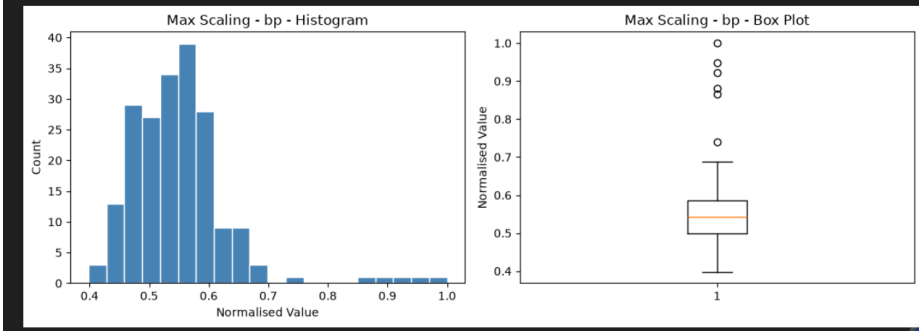
```python
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_max, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Max Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_max)
ax2.set_title("Max Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



B.4 Min-Max Scaling ((x − min) / (max − min))

B9. Apply min-max scaling to both columns. Confirm that every minimum is 0 and every maximum is 1.

```python
bp_mm = (df["bp"] - df["bp"].min()) / (df["bp"].max() - df["bp"].min())
wait_mm = (df["wait"] - df["wait"].min()) / (df["wait"].max() - df["wait"].min())

print("BP Min-Max Scaling")
print("Min:", bp_mm.min())
print("Max:", bp_mm.max())

print("\nWait Min-Max Scaling")
print("Min:", wait_mm.min())
print("Max:", wait_mm.max())
```

✓ 0.0s

```
BP Min-Max Scaling
Min: 0.0
Max: 1.0

Wait Min-Max Scaling
Min: 0.0
Max: 1.0
```

B10. Plot histogram + box plot for each column. In a markdown cell, locate the outlier dots on the box plots: do they look more squashed towards the high end compared with the z-score and IQR plots you will draw later? (Make a note — you will return to this in B.7.)

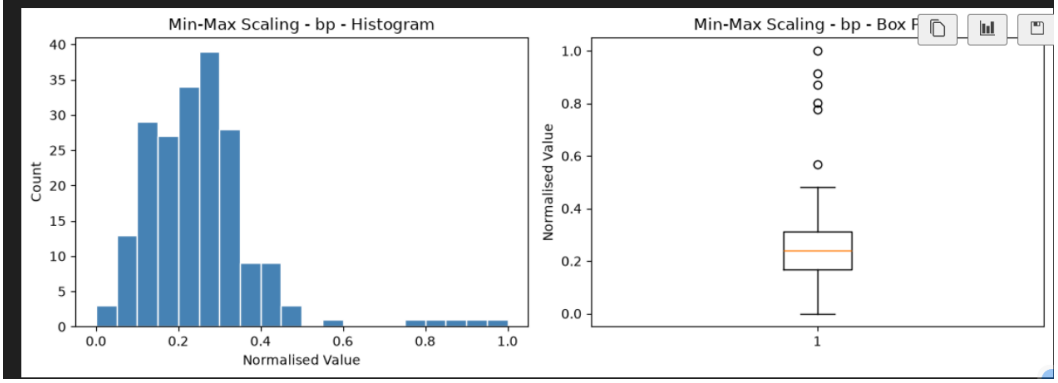
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_mm, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Min-Max Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_mm)
ax2.set_title("Min-Max Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



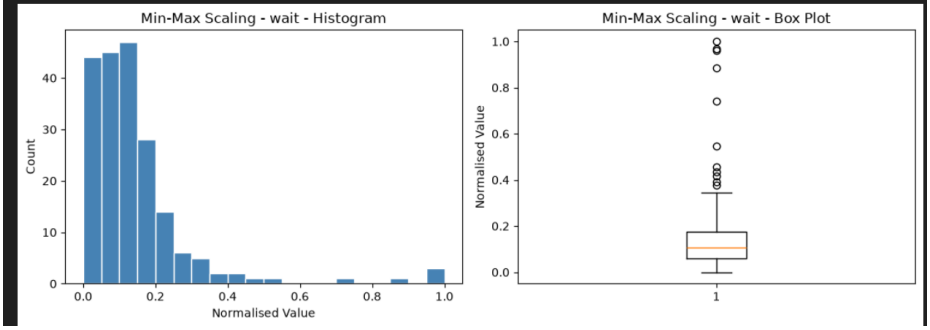

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(wait_mm, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Min-Max Scaling - wait - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(wait_mm)
ax2.set_title("Min-Max Scaling - wait - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



B11. Why is Min-Max sensitive to outliers?

Min-Max scaling is sensitive to outliers because it uses the minimum and maximum values of the dataset. If a value of 10,000 is added to the `wait` column, the maximum becomes 10,000. This greatly increases the range, causing most of the non-outlier values to be compressed very close to 0 after scaling.

B.5 Z-Score Normalisation ((x − mean) / std)

B12. Apply z-score normalisation to both columns. Print the mean and standard deviation of each result — they should be approximately 0 and 1. Are they exactly 0 and 1? Why might there be a tiny numerical difference?

```
bp_z = (df["bp"] - df["bp"].mean()) / df["bp"].std()
wait_z = (df["wait"] - df["wait"].mean()) / df["wait"].std()

print("BP")
print("Mean:", bp_z.mean())
print("Std :", bp_z.std())

print("\nWait")
print("Mean:", wait_z.mean())
print("Std :", wait_z.std())
```

✓ 0.0s

```
BP
Mean: 3.7747582837255325e-17
Std : 0.9999999999999999

Wait
Mean: 8.881784197001253e-18
Std : 1.0
```

B13. Plot histogram + box plot for each column. In a markdown cell, identify the z-score value of the outliers from the box plot (read off the approximate y-position of the outlier dots). Are any beyond ±3? What does a z-score of +4 mean in plain language?

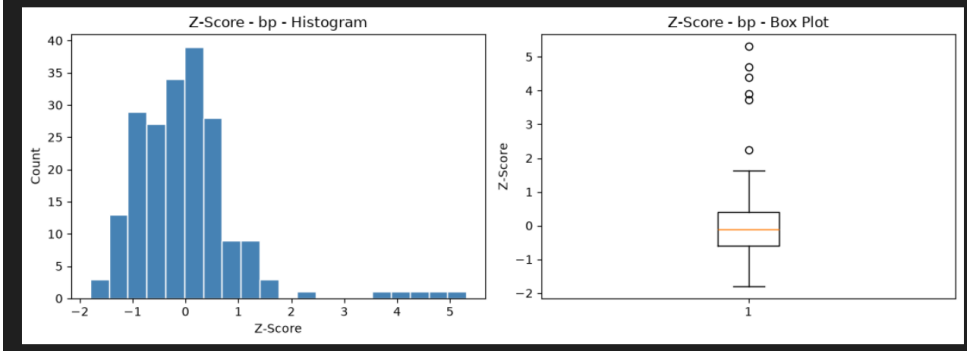
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.hist(bp_z, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Z-Score - bp - Histogram")
ax1.set_xlabel("Z-Score")
ax1.set_ylabel("Count")

ax2.boxplot(bp_z)
ax2.set_title("Z-Score - bp - Box Plot")
ax2.set_ylabel("Z-Score")

plt.tight_layout()
plt.show()
```

✓ 0.1s



B14. Compare the z-score histogram of bp with its raw histogram from B.1. Write one sentence: how did the shape change, and how did the x-axis scale change?

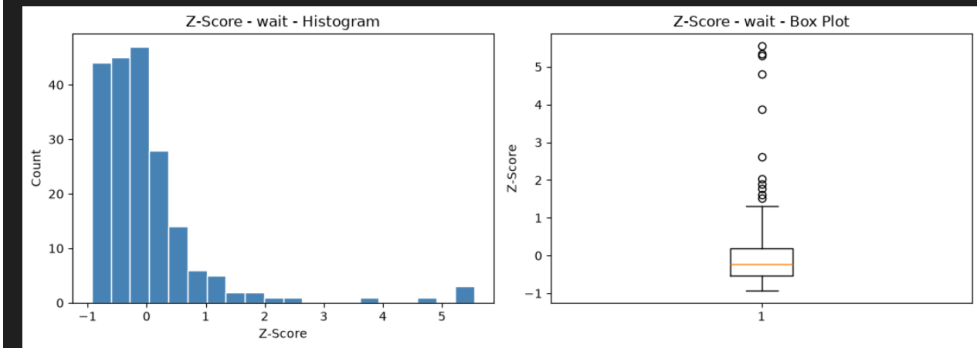
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.hist(wait_z, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Z-Score - wait - Histogram")
ax1.set_xlabel("Z-Score")
ax1.set_ylabel("Count")

ax2.boxplot(wait_z)
ax2.set_title("Z-Score - wait - Box Plot")
ax2.set_ylabel("Z-Score")

plt.tight_layout()
plt.show()
```

✓ 0.1s



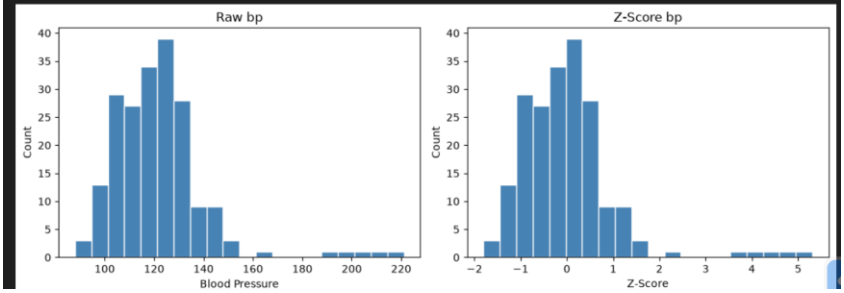

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Raw bp
ax1.hist(df["bp"], bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Raw bp")
ax1.set_xlabel("Blood Pressure")
ax1.set_ylabel("Count")

# Z-score bp
ax2.hist(bp_z, bins=20, color="steelblue", edgecolor="white")
ax2.set_title("Z-Score bp")
ax2.set_xlabel("Z-Score")
ax2.set_ylabel("Count")

plt.tight_layout()
plt.show()
```

✓ 0.5s



B.6 IQR / Robust Scaling ((x − median) / IQR)

B15. Apply IQR scaling to both columns. Print the median and IQR of each result — the median of the normalised column should be exactly 0. Verify this.

```
bp_median = df["bp"].median()
bp_q1 = df["bp"].quantile(0.25)
bp_q3 = df["bp"].quantile(0.75)
bp_iqr = (df["bp"] - bp_median) / (bp_q3 - bp_q1)

wait_median = df["wait"].median()
wait_q1 = df["wait"].quantile(0.25)
wait_q3 = df["wait"].quantile(0.75)
wait_iqr = (df["wait"] - wait_median) / (wait_q3 - wait_q1)

print("BP")
print("Median:", bp_iqr.median())
print("IQR:", round(bp_iqr.quantile(0.75) - bp_iqr.quantile(0.25), 4))

print("\nWait")
print("Median:", wait_iqr.median())
print("IQR:", round(wait_iqr.quantile(0.75) - wait_iqr.quantile(0.25), 4))
```

✓ 0.0s

```
BP
Median: -3.7339922820400773e-16
IQR: 1.0

Wait
Median: 0.0
IQR: 1.0
```

B16. Plot histogram + box plot for each column. In a markdown cell, observe the outlier dots on the box plots: are they farther from the box compared with z-score normalisation? Explain why IQR scaling pushes outliers further out while keeping the main body of the data more compact.

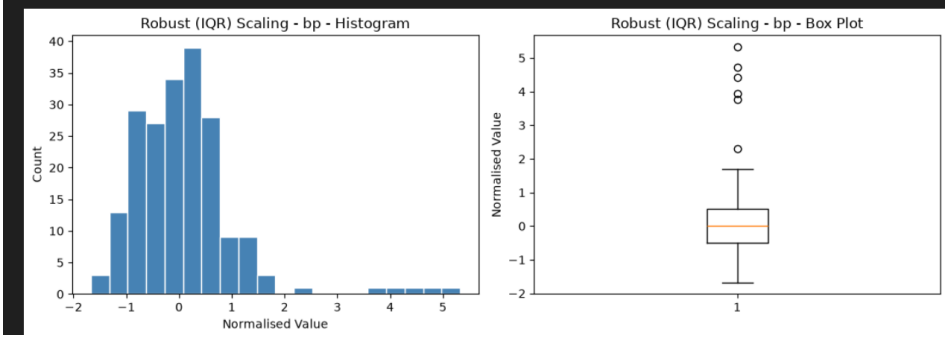
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(bp_iqr, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Robust (IQR) Scaling - bp - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(bp_iqr)
ax2.set_title("Robust (IQR) Scaling - bp - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



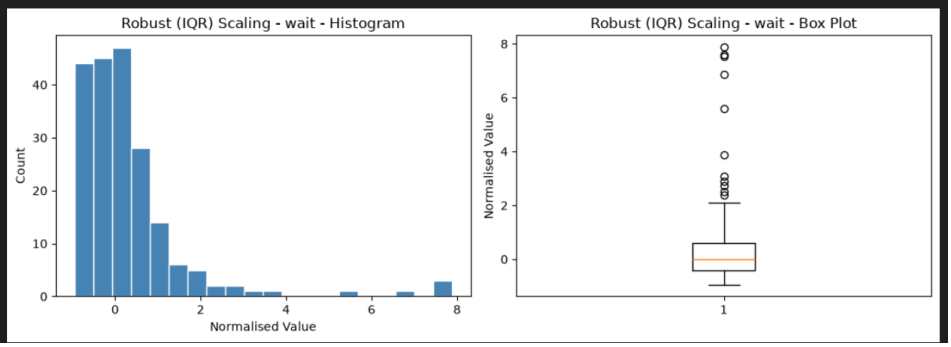
```python
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Histogram
ax1.hist(wait_iqr, bins=20, color="steelblue", edgecolor="white")
ax1.set_title("Robust (IQR) Scaling - wait - Histogram")
ax1.set_xlabel("Normalised Value")
ax1.set_ylabel("Count")

# Box Plot
ax2.boxplot(wait_iqr)
ax2.set_title("Robust (IQR) Scaling - wait - Box Plot")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.1s



B16. Observation

The IQR-scaled box plots show that the outliers can appear farther away from the main box compared with Z-score scaling. This happens because IQR is based on the middle 50% of the data and is less affected by extreme values. Therefore, the main body remains relatively compact while extreme observations receive larger scaled values.

```python
wait_z = (df["wait"] - df["wait"].mean()) / df["wait"].std()

print(wait_z.head())
```

✓ 0.0s

```
0   -0.110237
1   -0.251626
2   -0.897215
3   -0.647783
4   -0.658454
Name: wait, dtype: float64
```

B17. Compare the IQR-scaled box plot of wait with the z-score-scaled box plot of wait side by side. In a markdown cell, state which method you would prefer for a dataset with many extreme outliers and give one reason.

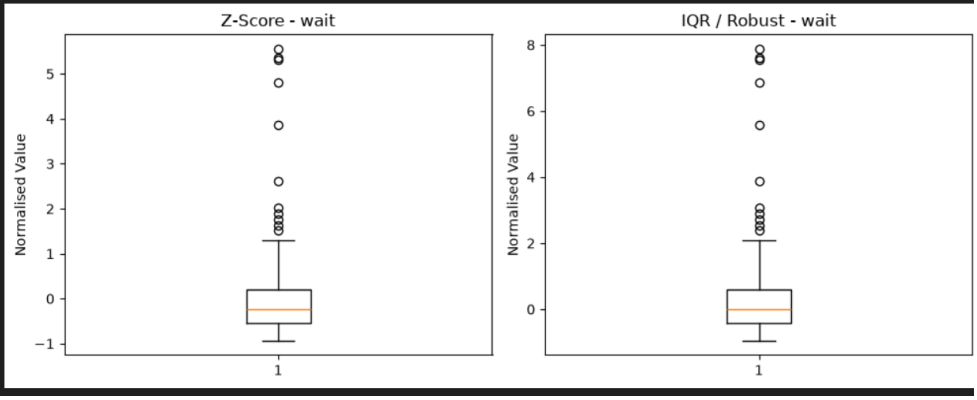
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

# Z-score
ax1.boxplot(wait_z)
ax1.set_title("Z-Score - wait")
ax1.set_ylabel("Normalised Value")

# IQR
ax2.boxplot(wait_iqr)
ax2.set_title("IQR / Robust - wait")
ax2.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.0s



B.7 Summary Comparison

B18. Create a single DataFrame containing the raw column and all five normalised versions of bp. Call describe() on it and print the result. In a markdown cell, fill in this observation table: Technique | min | max | mean | median raw | ? | ? | ? | ? min scale | ... | ... | ... | ... (and so on for all five) Which two techniques produce the same shape with different x-axis scales?

```
bp_comparison = pd.DataFrame({
    "raw": df["bp"],
    "min_scale": bp_min,
    "max_scale": bp_max,
    "min_max": bp_mm,
    "z": bp_z,
    "iqr": bp_iqr
})

bp_comparison.describe().round(3)
```

✓ 0.0s

	raw	min_scale	max_scale	min_max	z	iqr
count	200.000	200.000	200.000	200.000	200.000	200.000
mean	121.723	33.723	0.551	0.253	0.000	0.101
std	18.717	18.717	0.085	0.141	1.000	0.984
min	88.000	0.000	0.398	0.000	-1.802	-1.671
25%	110.400	22.400	0.499	0.168	-0.605	-0.494
50%	119.800	31.800	0.542	0.239	-0.103	-0.000
75%	129.425	41.425	0.585	0.311	0.411	0.506
max	221.100	133.100	1.000	1.000	5.309	5.325

B19. Draw one figure with five box plots side by side — one per normalisation technique — for the wait column. Use plt.subplots(1,5) and label each subplot with the technique name. Write two sentences: which technique most compresses the main body of the data, and which technique makes outliers look the most extreme?

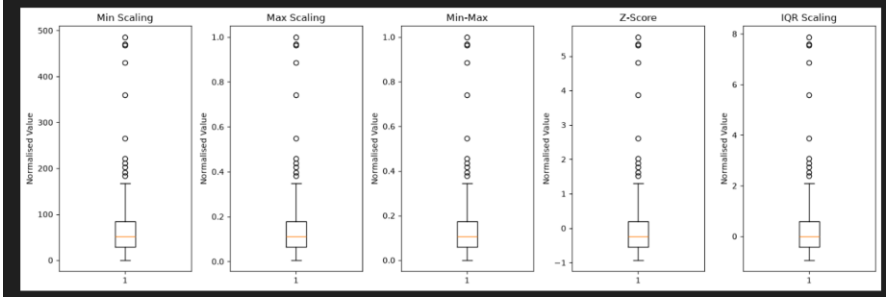
```
fig, axes = plt.subplots(1, 5, figsize=(15, 5))

data = [wait_min, wait_max, wait_mm, wait_z, wait_iqr]
titles = ["Min Scaling", "Max Scaling", "Min-Max", "Z-Score", "IQR Scaling"]

for ax, values, title in zip(axes, data, titles):
    ax.boxplot(values)
    ax.set_title(title)
    ax.set_ylabel("Normalised Value")

plt.tight_layout()
plt.show()
```

✓ 0.2s



B20. In a final markdown cell, write a short paragraph (4–6 sentences) answering: for a machine learning model that is sensitive to the scale of features (e.g., K-NN), which normalisation technique would you choose for a dataset that contains outliers, and why? Mention at least two techniques in your comparison.

B20. Final Comparison

For a scale-sensitive machine learning model such as K-NN, I would choose IQR/Robust Scaling when the dataset contains outliers. It uses the median and IQR, so extreme values have less influence on the scaling. Z-score scaling is also useful, but it uses the mean and standard deviation, which can be affected by outliers. Min-Max scaling maps values between 0 and 1, but it is sensitive to extreme values because it depends on the minimum and maximum. Therefore, IQR/Robust Scaling is a better choice when the dataset contains many outliers.

B.8 Take-Home (Submit with Notebook)

B21. Generate a new column: bimodal = np.concatenate([rng.normal(50,5,100), rng.normal(150,5,100)]). Apply all five normalisations to it. For each, draw a histogram. Which normalisation preserves the bimodal shape most clearly? Which technique would be misleading if you assumed the output was always unimodal?

```
bimodal = np.concatenate([
    rng.normal(50, 5, 100),
    rng.normal(150, 5, 100)
])

bimodal_min = bimodal - bimodal.min()
bimodal_max = bimodal / bimodal.max()
bimodal_mm = (bimodal - bimodal.min()) / (bimodal.max() - bimodal.min())
bimodal_z = (bimodal - bimodal.mean()) / bimodal.std()

q1 = np.percentile(bimodal, 25)
q3 = np.percentile(bimodal, 75)
bimodal_iqr = (bimodal - np.median(bimodal)) / (q3 - q1)
```

0.0s

Generate + Code + Markdown

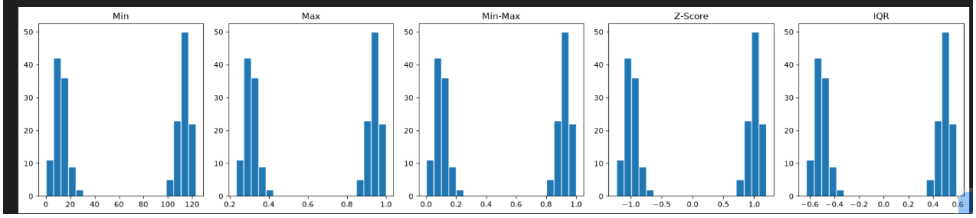
```
fig, axes = plt.subplots(1, 5, figsize=(18, 4))

data = [bimodal_min, bimodal_max, bimodal_mm, bimodal_z, bimodal_iqr]
titles = ["Min", "Max", "Min-Max", "Z-Score", "IQR"]

for ax, values, title in zip(axes, data, titles):
    ax.hist(values, bins=20, edgecolor="white")
    ax.set_title(title)

plt.tight_layout()
plt.show()
```

0.3s



B22. Without using any pandas or NumPy normalisation helper: write your own functions min_max_scale(arr) and z_score_scale(arr) using only basic Python arithmetic. Verify that their output matches your earlier pandas results for the bp column (hint: np.allclose is useful here).

```python
def min_max_scale(arr):
    minimum = min(arr)
    maximum = max(arr)

    return [(x - minimum) / (maximum - minimum) for x in arr]


def z_score_scale(arr):
    n = len(arr)
    mean = sum(arr) / n

    variance = sum((x - mean) ** 2 for x in arr) / (n - 1)
    std = variance ** 0.5

    return [(x - mean) / std for x in arr]
```

✓ 0.0s

```python
bp_manual_mm = min_max_scale(df["bp"].tolist())
bp_manual_z = z_score_scale(df["bp"].tolist())

print("Min-Max:", np.allclose(bp_manual_mm, bp_mm))
print("Z-Score:", np.allclose(bp_manual_z, bp_z))
```

✓ 0.0s

```
Min-Max: True
Z-Score: True
```

B23. Explore log transformation.

```python
log_wait = np.log1p(df["wait"])

print(log_wait.head())
```

✓ 0.0s

```
0    4.177459
1    4.000034
2    1.824549
3    3.214868
4    3.182212
Name: wait, dtype: float64
```